AGLC CRM Database Schema - Phase 1

Overview

This document defines the complete database schema for the AGLC Law Firm CRM system. The schema is designed using PostgreSQL and follows best practices for relational database design.

Core Tables

1. Users Table

Purpose: Store user account information and authentication data

```sql
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  openId VARCHAR(64) UNIQUE NOT NULL,
  email VARCHAR(320) UNIQUE,
  name TEXT,
  phone VARCHAR(20),
  role ENUM('admin', 'lawyer', 'staff', 'client') DEFAULT 'staff',
  department VARCHAR(100),
  title VARCHAR(100),
  avatar_url TEXT,
  bio TEXT,
  status ENUM('active', 'inactive', 'archived') DEFAULT 'active',
  loginMethod VARCHAR(64),
  lastSignedIn TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deletedAt TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_status ON users(status);
```

---

2. Clients Table

Purpose: Store client information and contact details

```sql
CREATE TABLE clients (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(320),
  phone VARCHAR(20),
  phone_alt VARCHAR(20),
  company_name VARCHAR(255),
  industry VARCHAR(100),
  country VARCHAR(100),
  city VARCHAR(100),
  address TEXT,
  vip_status BOOLEAN DEFAULT FALSE,
  vip_tier ENUM('bronze', 'silver', 'gold', 'platinum') DEFAULT NULL,
  contact_preference ENUM('email', 'phone', 'whatsapp', 'sms') DEFAULT
'email',
  communication_language ENUM('ar', 'en') DEFAULT 'ar',
  notes TEXT,
  tags JSON,
  metadata JSON,
  status ENUM('active', 'inactive', 'archived') DEFAULT 'active',
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deletedAt TIMESTAMP
);

CREATE INDEX idx_clients_email ON clients(email);
CREATE INDEX idx_clients_vip_status ON clients(vip_status);
CREATE INDEX idx_clients_status ON clients(status);
```

---

3. Cases Table

Purpose: Store legal case information and status

```sql
CREATE TABLE cases (
  id SERIAL PRIMARY KEY,
  case_number VARCHAR(50) UNIQUE NOT NULL,
  client_id INTEGER NOT NULL REFERENCES clients(id),
  title VARCHAR(255) NOT NULL,
  description TEXT,
  case_type VARCHAR(100),
  practice_area VARCHAR(100),
  status ENUM('intake', 'active', 'on_hold', 'closed', 'archived') DEFAULT
'intake',
  priority ENUM('low', 'medium', 'high', 'critical') DEFAULT 'medium',
  template_id INTEGER REFERENCES case_templates(id),
  lead_lawyer_id INTEGER REFERENCES users(id),
  assigned_to JSON,
  budget DECIMAL(12, 2),
  budget_spent DECIMAL(12, 2) DEFAULT 0,
  start_date DATE,
  expected_end_date DATE,
  actual_end_date DATE,
  phase ENUM('intake', 'discovery', 'negotiation', 'trial', 'appeal',
'closed') DEFAULT 'intake',
  phase_progress INTEGER DEFAULT 0,
  documents JSON,
  notes TEXT,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deletedAt TIMESTAMP
);

CREATE INDEX idx_cases_client_id ON cases(client_id);
CREATE INDEX idx_cases_lead_lawyer_id ON cases(lead_lawyer_id);
CREATE INDEX idx_cases_status ON cases(status);
CREATE INDEX idx_cases_priority ON cases(priority);
CREATE INDEX idx_cases_phase ON cases(phase);
```

---

4. Tasks Table

Purpose: Store individual tasks within cases

```sql
CREATE TABLE tasks (
  id SERIAL PRIMARY KEY,
  case_id INTEGER NOT NULL REFERENCES cases(id),
  title VARCHAR(255) NOT NULL,
  description TEXT,
  task_type ENUM('document', 'research', 'communication', 'meeting',
'filing', 'review', 'other') DEFAULT 'other',
  status ENUM('todo', 'in_progress', 'review', 'completed', 'blocked')
DEFAULT 'todo',
  priority ENUM('low', 'medium', 'high', 'critical') DEFAULT 'medium',
  assigned_to_id INTEGER REFERENCES users(id),
  created_by_id INTEGER REFERENCES users(id),
  due_date DATE,
  start_date DATE,
  completed_date DATE,
  estimated_hours DECIMAL(5, 2),
  actual_hours DECIMAL(5, 2),
  dependencies JSON,
  attachments JSON,
  comments TEXT,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deletedAt TIMESTAMP
);

CREATE INDEX idx_tasks_case_id ON tasks(case_id);
CREATE INDEX idx_tasks_assigned_to_id ON tasks(assigned_to_id);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_due_date ON tasks(due_date);
```

---

5. Case Templates Table

Purpose: Store reusable case templates

```sql
CREATE TABLE case_templates (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  practice_area VARCHAR(100),
  case_type VARCHAR(100),
  phases JSON,
  default_tasks JSON,
  documents JSON,
  created_by_id INTEGER REFERENCES users(id),
  is_active BOOLEAN DEFAULT TRUE,
  usage_count INTEGER DEFAULT 0,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE INDEX idx_case_templates_practice_area ON
case_templates(practice_area);
CREATE INDEX idx_case_templates_is_active ON case_templates(is_active);
```

---

6. Gamification Table

Purpose: Track user achievements and rewards

```sql
CREATE TABLE gamification_achievements (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  achievement_type VARCHAR(100),
  title VARCHAR(255),
  description TEXT,
  points INTEGER DEFAULT 0,
  badge_icon VARCHAR(255),
  achieved_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  metadata JSON
);

CREATE INDEX idx_gamification_user_id ON gamification_achievements(user_id);
CREATE INDEX idx_gamification_achieved_at ON
gamification_achievements(achieved_at);
```

---

7. Rewards Table

Purpose: Store reward redemption history

```sql
CREATE TABLE rewards (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  reward_type VARCHAR(100),
  title VARCHAR(255),
  description TEXT,
  points_required INTEGER,
  points_redeemed INTEGER,
  status ENUM('available', 'redeemed', 'expired') DEFAULT 'available',
  redeemed_at TIMESTAMP,
  expires_at TIMESTAMP,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE INDEX idx_rewards_user_id ON rewards(user_id);
CREATE INDEX idx_rewards_status ON rewards(status);
```

---

8. Wellness Table

Purpose: Track wellness and health features

```sql
CREATE TABLE wellness_tracking (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  mood_score INTEGER,
  mood_notes TEXT,
  break_taken BOOLEAN DEFAULT FALSE,
  prayer_time_tracked BOOLEAN DEFAULT FALSE,
  water_intake INTEGER,
  exercise_minutes INTEGER,
  sleep_hours DECIMAL(3, 1),
  stress_level INTEGER,
  tracked_date DATE DEFAULT CURRENT_DATE,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_wellness_user_id ON wellness_tracking(user_id);
CREATE INDEX idx_wellness_tracked_date ON wellness_tracking(tracked_date);
```

---

9. Communications Table

Purpose: Track all client communications

```sql
CREATE TABLE communications (
  id SERIAL PRIMARY KEY,
  case_id INTEGER REFERENCES cases(id),
  client_id INTEGER NOT NULL REFERENCES clients(id),
  sender_id INTEGER NOT NULL REFERENCES users(id),
  communication_type ENUM('email', 'phone', 'whatsapp', 'sms', 'meeting',
'letter') DEFAULT 'email',
  subject VARCHAR(255),
  content TEXT,
  direction ENUM('inbound', 'outbound') DEFAULT 'outbound',
  status ENUM('sent', 'received', 'read', 'archived') DEFAULT 'sent',
  attachments JSON,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE INDEX idx_communications_case_id ON communications(case_id);
CREATE INDEX idx_communications_client_id ON communications(client_id);
CREATE INDEX idx_communications_sender_id ON communications(sender_id);
```

---

10. Documents Table

Purpose: Store document metadata and references

```sql
CREATE TABLE documents (
  id SERIAL PRIMARY KEY,
  case_id INTEGER REFERENCES cases(id),
  task_id INTEGER REFERENCES tasks(id),
  document_name VARCHAR(255) NOT NULL,
  document_type VARCHAR(100),
  file_path VARCHAR(500),
  file_size INTEGER,
  mime_type VARCHAR(100),
  uploaded_by_id INTEGER REFERENCES users(id),
  version INTEGER DEFAULT 1,
  is_confidential BOOLEAN DEFAULT FALSE,
  tags JSON,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deletedAt TIMESTAMP
);

CREATE INDEX idx_documents_case_id ON documents(case_id);
CREATE INDEX idx_documents_task_id ON documents(task_id);
CREATE INDEX idx_documents_uploaded_by_id ON documents(uploaded_by_id);
```

---

11. AI Agent Logs Table

Purpose: Track AI agent activities and decisions

```sql
CREATE TABLE ai_agent_logs (
  id SERIAL PRIMARY KEY,
  agent_type VARCHAR(100),
  case_id INTEGER REFERENCES cases(id),
  task_id INTEGER REFERENCES tasks(id),
  action_type VARCHAR(100),
  input_data JSON,
  output_data JSON,
  decision_reason TEXT,
  confidence_score DECIMAL(3, 2),
  execution_time_ms INTEGER,
  status ENUM('success', 'partial', 'failed') DEFAULT 'success',
  error_message TEXT,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_ai_logs_agent_type ON ai_agent_logs(agent_type);
CREATE INDEX idx_ai_logs_case_id ON ai_agent_logs(case_id);
CREATE INDEX idx_ai_logs_createdAt ON ai_agent_logs(createdAt);
```

---

12. Notifications Table

Purpose: Store system notifications

```sql
CREATE TABLE notifications (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  notification_type VARCHAR(100),
  title VARCHAR(255),
  message TEXT,
  related_case_id INTEGER REFERENCES cases(id),
  related_task_id INTEGER REFERENCES tasks(id),
  is_read BOOLEAN DEFAULT FALSE,
  read_at TIMESTAMP,
  action_url VARCHAR(500),
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notifications_user_id ON notifications(user_id);
CREATE INDEX idx_notifications_is_read ON notifications(is_read);
CREATE INDEX idx_notifications_createdAt ON notifications(createdAt);
```

---

13. Analytics Table

Purpose: Store analytics and performance metrics

```sql
CREATE TABLE analytics_events (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id),
  event_type VARCHAR(100),
  event_name VARCHAR(255),
  case_id INTEGER REFERENCES cases(id),
  task_id INTEGER REFERENCES tasks(id),
  event_data JSON,
  duration_ms INTEGER,
  metadata JSON,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_analytics_user_id ON analytics_events(user_id);
CREATE INDEX idx_analytics_event_type ON analytics_events(event_type);
CREATE INDEX idx_analytics_createdAt ON analytics_events(createdAt);
```

Relationships Diagram

```
users (1) ──────────────── (many) cases (lead_lawyer)
users (1) ──────────────── (many) tasks (assigned_to)
users (1) ──────────────── (many) communications (sender)
users (1) ──────────────── (many) gamification_achievements
users (1) ──────────────── (many) wellness_tracking

clients (1) ──────────────── (many) cases
clients (1) ──────────────── (many) communications

cases (1) ──────────────── (many) tasks
cases (1) ──────────────── (many) documents
cases (1) ──────────────── (many) communications
cases (1) ──────────────── (many) ai_agent_logs

tasks (1) ──────────────── (many) documents
tasks (1) ──────────────── (many) ai_agent_logs

case_templates (1) ──────────────── (many) cases
```

Indexing Strategy

Primary Indexes (Performance Critical)

- `users.openId` - Authentication lookups
- `cases.client_id` - Case retrieval by client
- `tasks.case_id` - Task retrieval by case
- `tasks.assigned_to_id` - Task retrieval by assignee
- `communications.case_id` - Communication retrieval by case

Secondary Indexes (Query Optimization)

- `cases.status` - Case filtering
- `tasks.status` - Task filtering

- `tasks.due_date` - Task sorting and filtering
- `notifications.user_id` - Notification retrieval
- `analytics_events.createdAt` - Analytics time-based queries

Data Integrity Constraints

Foreign Key Constraints

- All `*_id` fields reference their parent tables
- Cascading deletes configured where appropriate
- Soft deletes used for audit trail

Unique Constraints

- `users.openId` - Unique per user
- `clients.email` - Unique per client
- `cases.case_number` - Unique per case

Check Constraints

- Status enums validated at database level
- Priority levels constrained
- Date validations (start_date < end_date)

Migration Strategy

Phase 1 Migrations

1. Create base tables (users, clients, cases, tasks)
2. Create supporting tables (templates, documents)
3. Create tracking tables (communications, gamification)

4. Create analytics tables (ai_agent_logs, analytics_events)

Data Seeding

- Default case templates
- Default users (admin, demo users)
- Sample clients (for testing)
- Sample cases (for testing)

Performance Considerations

Query Optimization

- Indexes on frequently filtered columns
- Composite indexes for common joins
- Denormalization where necessary for performance

Scalability

- Partitioning strategy for large tables (cases, tasks)
- Archive tables for historical data
- Read replicas for reporting queries

Backup Strategy

- Daily full backups
- Hourly incremental backups
- Point-in-time recovery capability
- Backup verification and testing

Security Measures

Data Protection

- Encryption at rest for sensitive data
- Encryption in transit (TLS)
- Row-level security (RLS) for multi-tenant data
- Audit logging for all changes

Access Control

- Role-based access control (RBAC)
- Column-level permissions
- Sensitive data masking
- Activity logging

Maintenance

Regular Tasks

- Index fragmentation analysis
- Query performance monitoring
- Backup verification
- Archive old data

Monitoring

- Query execution times
- Index usage statistics
- Table growth rates
- Connection pool usage

Schema Version: 1.0
Last Updated: March 12, 2026
Database: PostgreSQL 14+
Status: Ready for Implementation